

Многофайловые программы

1. Заголовки и файлы с исходным кодом

Ещё в конце прошлого семестра вы начали сталкиваться с тем, что коды стали весьма внушительной длины, и ориентироваться с ними стало сложно. На этом месте пора переходить к программам, состоящим из нескольких файлов.

Когда появляются классы, их принято выносить в отдельные файлы. На это есть несколько причин:

- 1) классы могут быть объёмны и могут затруднять ориентирование в основном коде программы;
- 2) классы являются обособленными единицами и не зависят от кода основной программы;
- 3) классы могут использоваться не только в файле с основной программой, но и в файлах с описанием других классов.

Причина № 3, кроме вынесения класса из основного файла, приводит к разделению кода класса на 2 файла:

- 1) файл с объявлением класса его методов – заголовочный файл («хедер», header) – файл с расширением .h (иногда .hpp или .h++)
- 2) файл с определением методов класса – файл с исходным кодом («сорс», source) – файл с расширением .cpp (иногда .c++)

Расширения .h у вас, несомненно уже всплывали, некоторые из вас в прошлом семестре даже подключали устаревшие заголовки библиотек, пришедших из языка C в C++, которые имели это расширение.

Аналогичным образом с помощью директивы `#include` мы будем подключать и заголовочные файлы наших классов, только с одной небольшой поправкой – вместо треугольных скобок (`<` и `>`) будут использоваться кавычки. Это говорит о том, что подключается пользовательский файл, который не стоит искать в местах расположения стандартных библиотек.

На данном этапе рекомендую создавать файлы через интерфейс вашей среды разработки. В этом случае они будут добавляться в конфигурацию и не будет проблем при сборке.

Что же должно содержаться в заголовочном файле? В первую очередь надо обратить внимание на защиту от повторного «подключения» файла. Вы же помните, что директива `#include` просто копирует содержимое подключаемого файла? Есть 2 способа защитить файл от повторного подключения:

- 1) использовать **header guards** – набор директив препроцессора для условной компиляции. Тогда файл будет начинаться с `#ifndef UNIQUE_CODE_NAME`, где на месте `UNIQUE_CODE_NAME` должно стоять некоторое уникальное имя,

следующей строкой – `#define UNIQUE_CODE_NAME`, а последней строкой – `#endif`. Работает буквально следующим образом: если не объявлен идентификатор `UNIQUE_CODE_NAME`, то он объявляется вместе со следующим за ним кодом до директивы `#endif`.

- 2) Использовать директиву `#pragma`. Тогда первой строкой в файле указывается `#pragma once`, и на этом особенности заканчиваются.

В **каждом** заголовочном файле должна быть защита в соответствии с **одним** из вариантов выше.

Плюс первого варианта – это стандарт. Минус – надо придумывать уникальное имя, чтобы не было коллизий (обычно используется имя файла с `_N` на конце).

Плюс второго варианта – простота, не нужно ничего придумывать. Минус – не входит в стандарт C++, но поддерживается большинством компиляторов. И, пожалуй, это единственный раз, когда я могу порекомендовать нестандартщину.

Итак, в хедер выносятся объявления класса с объявлениями (прототипами функций). В прототипах функций можно опускать названия параметров, оставляя только их типы, *но я не рекомендую этого делать*. Кроме того, в хедере могут быть определены маленькие функции – например, геттеры и сеттеры. В примере 1 приведено содержание заголовочного файла для класса `Matrix`. Для простоты ориентирования названия файлов, в которых описан класс, обычно совпадают с названием класса.

Пример 1. Содержание файла `Matrix.h`

```
1  #pragma once
2
3  class Matrix
4  {
5  private:
6      int n, m;
7      float** values;
8  public:
9      Matrix() : n(0), m(0), values(nullptr) {} // конструктор по умолчанию
10     Matrix(const int &n_, const int &m_) : n(n_), m(m_) {}
11     // конструкторы с параметрами
12     Matrix(const int &n_, const int &m_, float** const& values_);
13     Matrix(const Matrix &matrix); // конструктор копирования
14     Matrix transpose(); // транспонирование матрицы
15     int getRows() // геттер кол-ва строк
16     {
17         return n;
18     }
19     int getColumns() // геттер кол-ва столбцов
20     {
21         return m;
22     }
23     void setDimentions(const int& n_, const int& m_); // сеттер размера
24     void fillValues(); // заполнение матрицы
25     void print(); // вывод матрицы на экран
26     Matrix& operator=(const Matrix &matrix); // присваивание копированием
27     Matrix operator+(const Matrix &matrix); // сложение матриц
28     ~Matrix(); // деструктор
29 };
```

Пример 2. Содержание файла Matrix.cpp

```
1  #include "Matrix.h"
2  #include <limits>
3  #include <ctime>
4  #include <iostream>
5
6  // конструктор с параметрами
7  Matrix::Matrix(const int &n_, const int &m_, float** const& values_)
8                                     : n(n_), m(m_)
9  {
10     values = new float*[n];
11     for (size_t i = 0; i < n; ++i)
12     {
13         values[i] = new float[m];
14         for (size_t j = 0; j < m; ++j)
15             values[i][j] = values_[i][j];
16     }
17 }
18
19 // конструктор копирования
20 Matrix::Matrix(const Matrix &matrix) : n(matrix.n), m(matrix.m)
21 {
22     if (matrix.values != nullptr)
23     {
24         values = new float*[n];
25         for (int i = 0; i < n; ++i)
26         {
27             values[i] = new float[m];
28             for (int j = 0; j < m; ++j)
29                 values[i][j] = matrix.values[i][j];
30         }
31     }
32 }
33
34 // транспонирование матрицы
35 Matrix Matrix::transpose()
36 {
37     Matrix newMatrix(m, n);
38     newMatrix.values = new float*[m];
39     for (size_t i = 0; i < m; ++i)
40     {
41         newMatrix.values[i] = new float[n];
42         for (size_t j = 0; j < n; ++j)
43             newMatrix.values[i][j] = values[j][i];
44     }
45     return newMatrix;
46 }
47
48 // сеттер размерности
49 void Matrix::setDimentions(const int& n_, const int& m_)
50 {
51     if ((n_ > 0) && (m_ > 0) && (values == nullptr))
52     {
53         n = n_;
54         m = m_;
55     }
56 }
57
```

Продолжение примера 2. Содержание файла Matrix.cpp

```
58 // заполнение матрицы
59 void Matrix::fillValues()
60 {
61     if ((n == 0) || (m == 0))
62         return;
63
64     std::srand(std::time(nullptr));
65     if (values == nullptr)
66     {
67         values = new float*[n];
68         for (size_t i = 0; i < n; ++i)
69             values[i] = new float[m];
70     }
71     for (size_t i = 0; i < n; ++i)
72         for (size_t j = 0; j < m; ++j)
73             values[i][j] = rand() % 201 - 100;
74 }
75
76 // вывод матрицы на экран
77 void Matrix::print()
78 {
79     if (values != nullptr)
80         for (size_t i = 0; i < n; ++i)
81         {
82             for (size_t j = 0; j < m; ++j)
83             {
84                 std::cout.width(4);
85                 std::cout << values[i][j] << " ";
86             }
87             std::cout << std::endl;
88         }
89 }
90
91 // оператор присваивания копированием
92 Matrix& Matrix::operator=(const Matrix &matrix)
93 {
94     if (this == &matrix)
95         return *this;
96
97     if (values != nullptr)
98     {
99         for (size_t i = 0; i < n; ++i)
100             delete[] values[i];
101         delete[] values;
102     }
103
104     n = matrix.n;
105     m = matrix.m;
106     if (matrix.values != nullptr)
107     {
108         values = new float*[n];
109         for (int i = 0; i < n; ++i)
110         {
111             values[i] = new float[m];
112             for (int j = 0; j < m; ++j)
113                 values[i][j] = matrix.values[i][j];
114         }
115     }
116
117     return *this;
118 }
119
```

Продолжение примера 2. Содержание файла Matrix.cpp

```
120 // сложение матриц
121 Matrix Matrix::operator+(const Matrix &matrix)
122 {
123     Matrix newMatrix;
124     if ((n != matrix.n) || (m != matrix.m))
125         return newMatrix;
126     newMatrix = *this;
127     for (size_t i = 0; i < n; ++i)
128         for (size_t j = 0; j < m; ++j)
129             newMatrix.values[i][j] += matrix.values[i][j];
130     return newMatrix;
131 }
132
133 // деструктор
134 Matrix::~~Matrix()
135 {
136     if (values != nullptr)
137     {
138         for (size_t i = 0; i < n; ++i)
139             delete[] values[i];
140         delete[] values;
141     }
142 }
```

А вот основной файл программы, где содержится main, остался очень кратким и понятным.

Пример 3. Содержание файла Source.cpp

```
1  #include <iostream>
2  #include "Matrix.h"
3
4  int main()
5  {
6      Matrix a(3,4),
7             b(3,4);
8      a.fillValues();
9      b.fillValues();
10     std::cout << "Matrix a:\n";
11     a.print();
12     std::cout << "\nMatrix b:\n";
13     b.print();
14     std::cout << "\nMatrix a + b:\n";
15     (a + b).print();
16     b = a;
17     std::cout << "\nMatrix 2*a:\n";
18     (a + b).print();
19     std::cout << "\nMatrix a (transposed):\n";
20     a.transpose().print();
21     return 0;
22 }
```

Обращаю внимание, что подключается только заголовочный файл. Назначение заголовочного файла – «познакомить» компьютер с имеющимися типами данных и функциями. На этапе препроцессорной обработки в Source.cpp вставляется содержимое Matrix.h, а затем оба файла .cpp компилируются. Их связывание в единый исполняемый файл производится на самом

последнем шаге – линковке. Если вы забыли о том, как исходный код превращается в исполняемый файл, то вспоминайте материал первого семинара прошлого семестра.

2. Почему многофайловые программы – это хорошо

Для пользующихся средами разработки процесс компиляции невидим, но он есть и для больших проектов он достаточно трудоёмок. Разбиение программы на отдельные файлы с исходным кодом позволяет отдельно компилировать только те части проекта, в которые были внесены изменения.

Обращаю внимание, что это справедливо для файлов с исходным кодом (.c, .cpp, .c++), т.к. именно они превращаются в объектные и подвергаются линковке. Вынесение части кода в заголовочный файл даст вам только сокращение размера основного файла с исходным кодом, и больше никакой практической пользы.

Поэтому если вы работаете с составной программой, в которой много функций, но изменения в которых происходят редко, выносите их объявления в заголовочный файл, а определения (реализации) – в отдельный файл с исходным кодом.

3. Предварительно скомпилированные заголовки

Некоторые из вас, вероятно, уже сталкивались с тем, что при создании непустых проектов среда разработки иногда требовала подключение странного пустого файла «stdafx.h» или «pch.h». Это предварительно скомпилированные заголовки – заголовочные файлы, которые были предварительно обработаны для оптимизации их использования при компиляции. В предварительно скомпилированные заголовки можно поместить те заголовки, которые меняются редко (и, соответственно, перекомпилировать каждый раз, когда они меняются) или не меняются вообще. Хороший пример последних – это заголовочные файлы стандартной библиотеки, которые вы точно ни разу за последний семестр не меняли ☺

Подключать предварительно скомпилированные заголовки нужно там, где они вам нужны. Если такой заголовок нужен только основному файлу проекта (где main), то подключайте только там. Если те же заголовки нужны вам в других файлах проекта, подключайте и там.

Зачастую в разных файлах требуются разные наборы библиотек. Универсального решения здесь нет, всегда есть 2 стула:

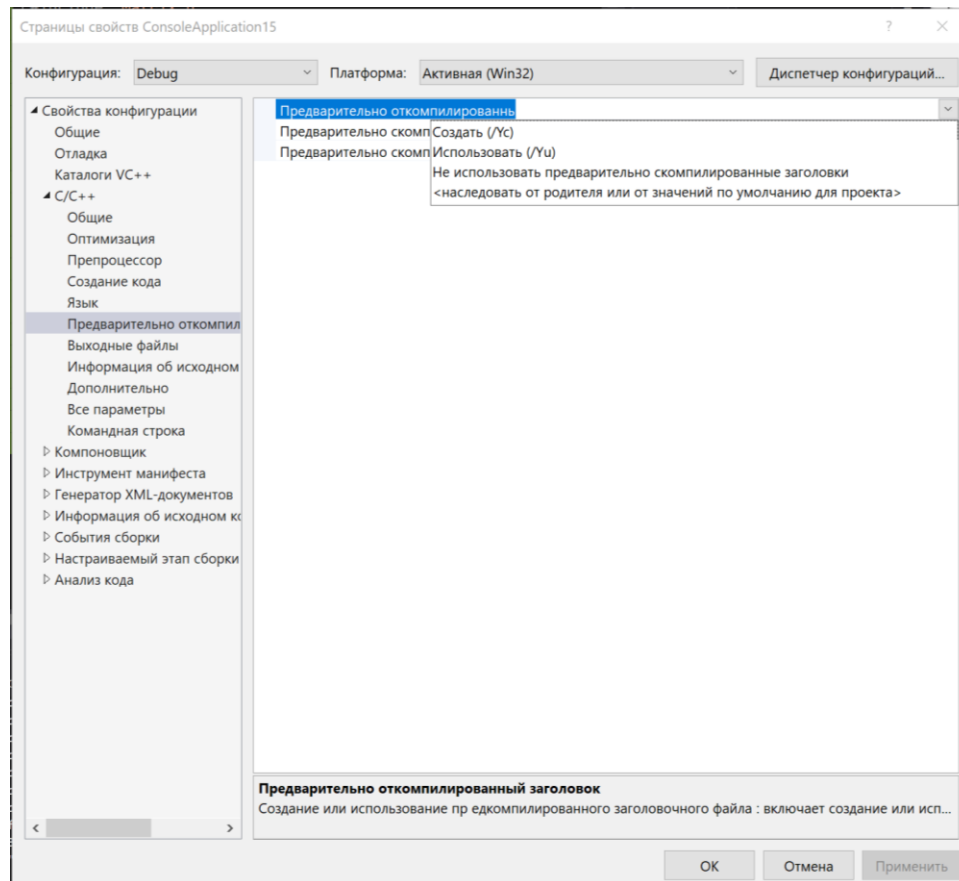
- 1) включать все библиотеки, делая избыточным для отдельно взятого файла исходного кода и тем самым увеличивая его размер;
- 2) включать только общие библиотеки, а несовпадающие подключать отдельно.

Какой из вариантов в отдельно взятом случае будет работать быстрее, зависит от случая.

Будьте внимательны! Использование или не использование предварительно откомпилированного заголовка – это настройка компилятора для каждого файла с исходным кодом. Если использование предварительно откомпилированного заголовка

выставлено для проекта в целом, будьте готовы, что среда разработки ожидает увидеть подключение заголовочного файла в каждом файле с исходным кодом, в т.ч. в файлах с реализациями классов.

Для файла с исходным кодом, который компилируется в файл предкомпилируемого заголовка, в настройках должно быть выставлено создание предварительно откомпилированного заголовка. Для других файлов, использующих заголовки – использование предварительно откомпилированных заголовков.



CLion ограниченно поддерживает предкомпилируемые заголовки, но иногда с ними бывают баги.